Trabalho 1 – Avaliação de Desempenho de uma Aplicação CRUD

Sistemas Distribuídos | Aluno: Heitor Argenta Preigschadt | Outubro de 2026

Parte 1 – Aplicação, Arquitetura e Cenários

Aplicação. Foi desenvolvido um CRUD de cadastro de pessoas em Python (Flask) com banco PostgreSQL. O banco só possuía uma tabela "pessoas" com os campos id, nome, email, telefone, cidade, data_cadastro e status. Os testes usaram dois endpoints: GET /api/pessoas (leitura) e POST /api/pessoas (escrita).

Arquitetura. Cliente–servidor em três camadas: os clientes (Apache JMeter) enviam requisições HTTP ao servidor web Flask, que acessa o PostgreSQL via driver psycopg2.

Dataset. O tabela foi populada com 100.000 registros gerados por um script em Python. Para a comparação pedida, também foi usada uma base reduzida de 20.000 registros.

Cenários. Cada teste usou 1.200 clientes simultâneos (ramp-up de 10s), divididos conforme foi pedido:

- A) 50% leitura / 50% escrita – 600 clientes lendo e 600 escrevendo;
- B) 75% leitura / 25% escrita – 900 lendo e 300 escrevendo;
- C) 25% leitura / 75% escrita – 300 lendo e 900 escrevendo.

Os três cenários foram executados em três configurações de banco: 100k com índices, 20k com índices e 100k sem índices, totalizando 9 execuções.

Parte 2 – Resultados

Base	Cenário	Erro leitura	Erro escrita	Erro total	Tempo médio	Vazão
100k com índice	A (50/50)	10,00%	10,67%	10,33%	1120 ms	106 req/s
	B (75/25)	14,11%	14,67%	14,25%	1066 ms	110 req/s
	C (25/75)	6,67%	7,33%	7,17%	1089 ms	108 req/s
20k com índice	A (50/50)	7,33%	9,50%	8,42%	1057 ms	105 req/s
	B (75/25)	10,44%	10,00%	10,33%	1058 ms	108 req/s
	C (25/75)	4,00%	3,89%	3,92%	1013 ms	108 req/s
100k sem índice	A (50/50)	7,00%	6,83%	6,92%	1092 ms	105 req/s
	B (75/25)	10,00%	9,33%	9,83%	1060 ms	109 req/s
	C (25/75)	8,00%	6,44%	6,83%	1059 ms	108 req/s

Comparação entre cenários. A vazão ficou praticamente igual (entre 105 e110 req/s) em todas as execuções e o tempo médio ficou em torno de 1 segundo. Isso indica que o limite está no servidor Flask, que atende as requisições de forma síncrona e abre uma conexão nova com o banco a cada requisição e não no tipo de operação. A diferença entre os cenários apareceu principalmente na taxa de erros, que foi maior no cenário B (mais leituras) e menor no cenário C (mais escritas).

Respostas às Perguntas

1. O que acontece se o dataset for reduzido para 20k registros?

O desempenho foi melhor. A taxa de erros caiu em todos os cenários (A: 10,33% para 8,42%; B: 14,25% para 10,33%; C: 7,17% para 3,92%) e o tempo médio diminuiu um pouco. Com menos dados, a tabela e os índices cabem melhor na memória do

PostgreSQL, então há menos acesso a disco. Já a vazão não mudou, pois ela é limitada pela estrutura utilizada na elaboração do sistema.

2. O que acontece com a adição de índices? Qual escolher? A consulta melhora?

Nos testes, as versões com índices tiveram mais erros que as versões sem índices na maioria dos cenários. Isso acontece nas escritas pois cada escrita agora precisa também atualizar os índices, tornando as escritas mais caras. Já as leituras não ganharam nada, o JMeter enviava o parâmetro cidade, que a API não utiliza da melhor forma, a API estava configurada de maneira a procurar usando ILIKE.

Portanto, a consulta não melhorou. O índice só traz benefício se a consulta e o índice forem adequados. Para buscar por cidade, o ideal seria a API filtrar com WHERE cidade = “cidade” e manter o índice em cidade. Para a busca textual da aplicação, usando ILIKE ‘%termo%’', um índice B-Tree não funciona.

3. Como a escolha do banco (SQL ou NoSQL) impacta o sistema?

O PostgreSQL é um banco SQL e isso garante as propriedades ACID além de trabalhar com esquema fixo, que talvez seja o mais adequado para um cadastro de pessoas. Em contrapartida, os bancos SQL escalam principalmente de forma vertical e o controle de transações fica muito caro quando há muita concorrência. Um banco NoSQL (MongoDB, Cassandra) costuma escrever os dados com mais facilidade e escalar horizontalmente com mais servidores de maneira mais fácil, mas frequentemente oferece consistência eventual e menos garantias. Para este sistema o PostgreSQL atende bem, um NoSQL só faria sentido com volume de escrita muito maior.

Conclusão

A aplicação suportou cerca de 107 requisições por segundo em todos os cenários, com taxa de erro entre 4% e 14%. O principal gargalo foi o servidor web síncrono e não o banco. Reduzir a base para 20k registros diminuiu os erros e os índices criados pioraram o desempenho porque aumentaram o custo das escritas sem serem usados nas leituras. Como melhorias, seria possível usar um servidor com vários workers (Gunicorn), um pool de conexões com o banco e índices adequados às consultas realmente feitas.